

4장

▼ 4.1

오늘날 인공 신경망에서 이용하는 구조(입력층, 은닉층, 가중치로 구성된 구조)는 **퍼셉트론**이라는 선형 분류기로, 퍼셉트론은 오늘날 신경망(딥러닝)의 기원이 되는 알고리즘

퍼셉트론은 다수의 신호를 입력으로 받아 하나의 신호를 출력하는데, 이 신호를 입력으로 받아 **흐른다/흐르지 않는다(1 또는 0)**라는 정보를 앞으로 전달하는 원리로 작동

AND 게이트

AND 게이트는 모든 입력이 1일 때 작동함.

→ 입력 중 어느 하나라도 0을 갖는다면 작동하지 않음.

X1	X2	y
0	0	0
1	0	0
0	1	0
1	1	1

→ AND 게이트는 선형적으로 분리할 수 있는 데이터

OR 게이트

OR 게이트는 입력에서 **하나만 또는 둘 다 1일 때** 작동함.

→ 입력 모두가 0인 경우를 제외한 나머지가 모두 1

X1	X2	y
0	0	0
1	0	1
0	1	1
1	1	1

→ OR 게이트 역시 데이터를 선형적으로 분류할 수 있음.

XOR 게이트

XOR 게이트는 **배타적 논리합**으로, 입력 두 개 중 **한 개만 1일 때** 작동하는 논리 연산

X1	X2	y
0	0	0
1	0	1
0	1	1
1	1	0

XOR 게이트의 데이터가 **비선형적으로 분리**되기 때문에 단층 퍼셉트론에서는 제대로 분류하기 어려움.

→ 단층 퍼셉트론에서는 AND, OR 연산에 대해서는 학습이 가능하지만 **XOR에 대해서는 학습이 불가능**

→ 입력층과 출력층 사이에 하나 이상의 **중간층(은닉층)**을 두어 비선형적으로 분리되는 데이터에 대해서도 학습할 수 있도록 **다층 퍼셉트론**이 고안됨.

입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망을 **심층 신경망(DNN, Deep Neural Network)**이라고 하며, 심층 신경망을 다른 이름으로 **딥러닝**이라고 함.

▼ 4.2

딥러닝은 여러 층을 가진 인공 신경망을 사용하여 학습을 수행하는 것

4.2.1 딥러닝 용어

딥러닝은 다음과 같이 **입력층, 출력층과 두 개 이상의 은닉층**으로 구성됨

또한 입력 신호를 전달하기 위해 다양한 함수가 사용됨

딥러닝을 구성하는 요소

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
층	은닉층(hidden layer)	모든 입력 노드로부터 입력값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 다음 층으로 전달하는 층
층	출력층(output layer)	신경망의 최종 결과가 포함된 층
	가중치(weight)	노드와 노드 간 연결 강도
	바이어스(bias)	가중합에 더해주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할

구분	구성 요소	설명
	가중합(weighted sum), 전달 함수	가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
함수	손실 함수(loss function)	가중치를 업데이트하기 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

가중치

가중치는 **입력값이 연산 결과에 미치는 영향력을 조절하는 요소**

예를 들어 어떤 가중치가 0 또는 0에 가까운 값이라면 입력값이 아무리 큰 값이라도 가중치와 곱한 값은 0이거나 0에 가까운 값이 됨.

가중합 또는 전달 함수

각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달하는데, 이 값들을 모두 더한 합계를 가중합이라고 함.

또한 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 **전달 함수**라고 함.

가중합은 다음과 같이 계산함.

$$\sum wx+b$$

- **w** : 가중치
- **x** : 입력값
- **b** : 바이어스

활성화 함수

활성화 함수는 전달 함수에서 전달받은 값을 출력할 때 **일정 기준에 따라 출력값을 변화시키는 비선형 함수**

시그모이드 함수

시그모이드 함수는 선형 함수의 결과를 **0~1 사이에서 비선형 형태로 변형**해 줌.

주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용됨.

과거에는 인기가 많았으나 딥러닝 모델의 깊이가 깊어지면 **기울기 소멸 문제(gradient vanishing problem)**가 발생하여 딥러닝 모델에서는 잘 사용하지 않음.

$$f(x) = 1 / (1 + e^{** (-x)})$$

기울기 소멸 문제 : 활성화 함수 계산 과정에서 도함수 값이 계속 작아지면서 가중치 변화가 0이 되어 더 이상 계산이 불가능한 상태

하이퍼볼릭 탄젠트 함수

하이퍼볼릭 탄젠트 함수는 선형 함수의 결과를 **-1~1 사이에서 비선형 형태로 변형**

시그모이드에서 출력값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는 데 사용했지만, **기울기 소멸 문제는 여전히 발생**

렐루 함수

입력값이 음수일 때는 0을 출력하고, 양수일 때는 입력값을 그대로 출력함

경사 하강법에 영향을 주지 않아 학습 속도가 빠르고, **기울기 소멸 문제가 발생하지 않는 장점이 있음**

렐루 함수는 일반적으로 **은닉층에서 사용**되며, 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠름

문제는 음수 값을 입력받으면 항상 0을 출력하기 때문에 학습 능력이 감소한다는 것

→ 이를 해결하려고 **리키 렐루(Leaky ReLU)** 함수가 사용됨

리키 렐루 함수

리키 렐루 함수는 입력값이 음수이면 0이 아닌 **0.001처럼 매우 작은 수를 반환**

→ 입력값이 수렴하는 구간이 제거되어 렐루 함수를 사용할 때 생기는 문제를 해결

소프트맥스 함수

소프트맥스(softmax) 함수는 입력값을 **0~1 사이에 출력되도록 정규화**하여 출력값들의 총합이 항상 1이 되도록 함

소프트맥스 함수는 보통 **딥러닝의 출력 노드의 활성화 함수**로 많이 사용됨

- 분자 → 입력 신호의 지수 함수
- 분모 → 모든 입력 신호의 지수 함수 합

즉, 각 출력값을 **확률처럼 표현**할 수 있음.

손실 함수

경사 하강법은 **학습률(α)**과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트하는 방법

→ 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동하는 방법
손실 함수는 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표

- 값이 클수록 → 많이 틀렸다는 의미
- 값이 0에 가까울수록 → 완벽하게 추정할 수 있다는 의미

평균 제곱 오차

실제값과 예측값의 차이를 제곱한 뒤 평균을 낸 것이 **평균 제곱 오차(MSE)**

실제값과 예측값의 차이가 클수록 평균 제곱 오차의 값도 커짐

→ 반대로 값이 작을수록 예측력이 좋다는 것을 의미

평균 제곱 오차는 **회귀에서 손실 함수로 주로 사용**

크로스 엔트로피 오차

크로스 엔트로피 오차(Cross Entropy Error)는 **분류 문제에서 원-핫 인코딩을 했을 때** 사용할 수 있는 오차 계산법

일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 시그모이드 함수를 사용함

하지만 시그모이드 함수에 포함된 자연 상수 e 때문에 평균 제곱 오차를 적용하면 매끄럽지 못한 그래프가 출력됨 → **크로스 엔트로피 손실 함수**를 사용

4.2.2 딥러닝 학습

딥러닝의 학습은 크게 **순전파**와 **역전파**라는 두 단계로 진행됨.

순전파

순전파(forward)는 네트워크에 훈련 데이터가 들어올 때 발생

데이터를 기반으로 예측값을 계산하기 위해 **전체 신경망을 거쳐 지나감**

즉,

입력 데이터 → 가중합 계산 → 활성화 함수 적용 → 다음 층으로 전달 → 은닉층을 거쳐 출력층까지 전달 → 최종적으로 예측값을 계산

네트워크를 통해 입력 데이터를 전달하며 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 예측값은 **출력층에 도달**하게 됨

그다음 손실 함수로 네트워크의 **예측값과 실제값의 차이(손실, 오차)**를 추정

손실 함수 비용은 **0이 이상적** → 손실 함수 비용이 0에 가까워지도록 모델이 훈련을 반복하면서 가중치를 조정

역전파

손실이 계산되면 그 정보는 **역으로 전파**되기 때문에 역전파(backpropagation)라고 함

출력층에서 시작된 손실 비용은 **출력층 → 은닉층 → 입력층** 순서로 전파됨

은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라 값이 달라짐

수학적으로 표현하면

- 예측값과 실제값의 차이를 각 뉴런의 가중치로 미분
- 기존 가중치 값에서 학습률을 곱하여 가중치 업데이트
- 출력층 → 은닉층 → 입력층 순서로 모든 뉴런에 대해 진행
- 계산된 각 뉴런 결과를 다시 순전파의 가중치 값으로 사용

즉, **순전파 → 손실 계산 → 역전파 → 가중치 업데이트 → 다시 순전파** 과정을 반복하면서 오차를 줄여감

4.2.3 딥러닝의 문제점과 해결 방안

딥러닝의 핵심은 **활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것**

은닉층 개수가 많을수록 데이터 분류가 잘되는 것을 볼 수 있음

하지만 은닉층이 많을수록 다음과 같은 문제점이 생김

과적합 문제 발생

과적합(over-fitting)은 **훈련 데이터를 과하게 학습해서 발생**

일반적으로 훈련 데이터는 실제 데이터의 일부분임.

따라서 훈련 데이터를 과하게 학습하면 훈련 데이터의 오차는 감소하지만 검증 데이터에 대해서는 오차가 증가할 수 있음

드롭아웃

신경망 모델이 과적합되는 것을 피하기 위한 방법

→ 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴.

드롭아웃을 사용하면 특정 노드에 지나치게 의존하여 학습하는 것을 방지할 수 있음.

기울기 소멸 문제

기울기 소멸 문제는 은닉층이 많은 신경망에서 주로 발생

출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 **학습이 되지 않는 현상**

즉, 기울기가 소멸하기 때문에 학습되는 값이 0에 가까워져 학습이 더디게 진행되다가 오차를 더 줄이지 못하고 그 상태로 수렴하는 현상임.

기울기 소멸 문제는 **시그모이드나 하이퍼볼릭 탄젠트 대신 렐루 활성화 함수 사용**으로 해결 가능

성능이 나빠지는 문제

경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동하는 과정을 반복

하지만 이 과정에서 성능이 나빠지는 문제가 발생할 수 있음.

이러한 문제점을 개선하고자

- 확률적 경사 하강법
- 미니 배치 경사 하강법

을 사용

배치 경사 하강법

→ 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트하는 방법

즉, 전체 훈련 데이터셋에 대해 가중치를 편미분함.

배치 경사 하강법은 모든 훈련 데이터셋을 사용하므로 **학습이 오래 걸리는 단점**이 있음.

확률적 경사 하강법

→ 임의로 선택한 데이터에 대해 기울기를 계산하는 방법

적은 데이터를 사용하므로 빠른 계산이 가능함.

하지만 파라미터 변경 폭이 불안정하고, 때로는 배치 경사 하강법보다 정확도가 낮을 수 있음.

→ 속도가 빠르다는 장점

미니 배치 경사 하강법

→ 전체 데이터셋을 **미니 배치**라는 여러 개의 그룹으로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트하면서 학습하는 방법.

전체 데이터를 계산하는 것보다 빠르며, 확률적 경사 하강법보다 안정적이라는 장점이 있음.

→ 실제로 가장 많이 사용

- BGD → 전체 데이터 사용
- SGD → 데이터 하나씩 사용
- Mini-batch → 데이터를 작은 묶음으로 나누어 사용

옵티마이저

확률적 경사 하강법의 **파라미터 변경 폭이 불안정한 문제**를 해결하기 위해 학습 속도와 운동량을 조정하는 **옵티마이저(optimizer)**를 적용할 수 있음

아다그라드

→ 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법

- 많이 변화하지 않는 변수 → 학습률을 크게
- 많이 변화하는 변수 → 학습률을 작게

많이 변화한 변수는 최적값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 값을 조정하고, 반대로 적게 변화한 변수는 학습률을 크게 하여 빠르게 오차를 줄이고자 함.

하지만 아다그라드는 **기울기가 0에 수렴하는 문제가 있어 사용하지 않으며**, 대신 알엠에 스프롭을 사용함.

아다델타

→ 아다그라드에서 **G값이 커짐에 따라 학습이 멈추는 문제**를 해결하기 위해 등장

아다그라드의 수식에서 학습률을 가중치의 변화량 크기를 누적인 값으로 변환했기 때문에 **학습률에 대한 하이퍼파라미터가 필요하지 않음.**

알엠에스프롭

→ 아다그라드의 G값이 무한히 커지는 것을 방지하고자 제안된 방법

아다그라드에서 학습이 안 되는 문제를 해결하기 위해 G 함수에 **지수 이동 평균**을 추가함.

→ G값이 너무 커지면 학습률이 작아져 학습이 안 될 수 있으므로 사용자가 감마 값을 이용하여 학습률 크기를 비율로 조정할 수 있도록 함.

운동량을 조정하는 방법

모멘텀

→ 경사 하강법과 마찬가지로 매번 기울기를 구하지만, 가중치를 수정하기 전에 **이전 수정 방향과 현재 수정 방향을 참고하여** 일정한 비율만 수정하는 방법.

수정이 양의 방향과 음의 방향으로 순차적으로 일어나는 **지그재그 현상이 줄어듦.**

이전 이동값을 고려하여 다음 값을 결정하므로 **관성 효과**를 얻을 수 있음.

→ SGD와 함께 사용

모멘텀 값은 0.9에서 시작하며 0.95, 0.99처럼 조금씩 증가시키면서 사용함.

네스테로프

→ 기존 모멘텀 방식은 기울기 값이 더해져 실제 가중치를 만드는 기존 SGD의 지점이 아닌 **새로운 지점에서 기울기를 계산함.**

모멘텀 방식은 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 단점이 있음.

네스테로프 방법은 **이동하기 전에 절반 정도 미리 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정함.**

→ 모멘텀의 빠른 이동 속도는 그대로 가져가면서 멈추어야 할 적절한 시점에서 제동을 걸기 쉬움.

속도와 운동량에 대한 혼용 방법

아담

→ 알엠에스프롭의 장점과 모멘텀의 장점을 결합한 경사 하강법

알엠에스프롭의 특징인 **지수 이동 평균**과 모멘텀의 속도 조절을 수식에 활용함.

→ 알엠에스프롭의 G 함수와 모멘텀의 함수를 사용하여 가중치를 업데이트

아담은 현재 가장 보편적으로 사용되는 옵티마이저 중 하나임.

4.2.4 딥러닝 사용 시 이점

특성 추출

컴퓨터가 입력받은 데이터를 분석하여 일정한 패턴이나 규칙을 찾아내려면 사람이 인지하는 데이터를 **컴퓨터가 인지할 수 있는 데이터로 변환**해 주어야 함.

이때 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 과정을 **특성 추출(feature extraction)**이라고 함.

딥러닝은 **특성 추출 과정을 알고리즘에 통합**함.

데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조를 사용하기 때문

빅데이터의 효율적 활용

딥러닝을 사용할 때의 이점으로 특성 추출이 있다고 했음.

즉, 딥러닝은 특성 추출을 알고리즘에 통합했는데, 이것이 가능한 이유는 **빅데이터** 때문

딥러닝 학습을 이용한 특성 추출은 **데이터 사례가 많을수록 성능이 향상**되기 때문

다른 말로 표현하면

→ 확보된 데이터가 적다면 딥러닝의 성능 향상을 기대하기 힘들

→ 데이터가 충분한지 고려한 후 머신러닝과 딥러닝을 선택해야 함.

▼ 4.3

딥러닝 알고리즘은 **심층 신경망을 사용**한다는 공통점이 있음.

목적에 따라

- 심층 신경망(DNN)

- 합성곱 신경망(CNN)
- 순환 신경망(RNN)
- 제한된 볼츠만 머신(RBM)
- 심층 신뢰 신경망(DBN)

등으로 분류됨.

4.3.1 심층 신경망

심층 신경망(DNN)은 **입력층과 출력층 사이에 다수의 은닉층을 포함하는** 인공 신경망
심층 신경망은 다수의 은닉층을 추가했기 때문에 **별도의 트릭 없이 비선형 분류가 가능**
함.

하지만

- 학습을 위한 연산량이 많음
- 기울기 소멸 문제가 발생할 수 있음

이러한 문제를 해결하고자 앞서 설명한

- 드롭아웃
- 렐루 함수
- 배치 정규화

등을 적용해야 함.

4.3.2 합성곱 신경망

→ **합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘**

영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데
유용

합성곱 신경망은 이미지에서 **객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 특히 유용**
함.

대표적인 합성곱 신경망

- LeNet-5
- AlexNet

층을 더 깊게 쌓은 신경망

- VGG
- GoogLeNet
- ResNet

기존 신경망과 비교한 CNN의 특징

- 각 층의 입출력 형상을 유지함
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용하기 때문에 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음

4.3.3 순환 신경망

→ 시계열 데이터처럼 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망

순환 신경망의 '순환'은 자기 자신을 참조한다는 것으로, 현재 데이터가 이전 결과와 연관이 있다는 의미.

순환 신경망의 특징

- 시간성(temporal property)을 가진 데이터가 많음
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
- 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음

순환 신경망은 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있음.

→ 이를 해결하고자 메모리 개념을 도입한 LSTM(Long Short-Term Memory)이 순환 신경망에서 많이 사용됨.

RNN의 활용

순환 신경망은 자연어 처리 분야와 궁합이 맞음.

4.3.4 제한된 볼츠만 머신

볼츠만 머신(Boltzmann machine)은 가시층과 은닉층으로 구성된 모델

이 모델에서 가시층은 은닉층과만 연결되는데, 가시층과 가시층 사이, 은닉층과 은닉층 사이에는 연결이 없음

제한된 볼츠만 머신의 특징

다음과 같은 곳에 사용함.

- 차원 감소
- 분류
- 선형 회귀 분석
- 협업 필터링
- 특성 값 학습
- 주제 모델링

→ 기울기 소멸 문제를 해결하기 위한 **사전 학습 용도**로 활용 가능

→ 심층 신뢰 신경망(DBN)의 기반으로 활용

4.3.5 심층 신뢰 신경망

→ 입력층과 은닉층으로 구성된 **제한된 볼츠만 머신**을 블록처럼 여러 층 쌓은 형태로 **연결된 신경망**

→ 사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조

레이블이 없는 데이터에 대한 **비지도 학습**이 가능

부분적인 이미지에서 전체를 연상하는 **일반화와 추상화 과정**을 구현할 때 사용하면 유용

심층 신뢰 신경망의 학습 절차

1. 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련
2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN 완성

심층 신뢰 신경망의 특징

- 순차적으로 심층 신뢰 신경망을 학습시키면서 계층적 구조를 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성을 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용

▼ 4.4

머신 러닝을 학습할지, 딥러닝을 학습할지는 주어진 데이터를 활용하여 어떤 결과를 얻고 싶은지에 따라 다름

간단한 선형 회귀나 분류를 이용하여 원하는 값을 도출할 수 있다면 머신 러닝만으로도 충분하지만 복잡한 비선형 데이터에 대한 분류 및 예측을 도출하고 싶다면 딥러닝을 학습해야 함.

머신 러닝과 딥러닝은 우리가 원하는 결과를 도출하기 위한 도구일 뿐이고, 어떤 도구를 선택했을 때 더 좋은 성능과 효과를 얻을 수 있을지는 데이터를 수집하고 분석하는 사람만 알 수 있음

데이터를 활용하여 얻고자 하는 것에 따라 머신 러닝 또는 딥러닝을 선택하고 학습 데이터를 훈련시키면 됨